# Domande quiz passati

Quali proprietà valgono per i value types in C#?

- ✓ a. Includono `struct`, `enum`, tipi numerici e `bool`
- b. Sono `null` di default
- c. Sono sempre allocati sullo heap come oggetti distinti
- ✓ d. L'assegnazione copia il valore

`run:` vs `uses:` in un step di GitHub Actions: quali affermazioni sono corrette?

- ✓ a. `run:` può essere usato per eseguire `dotnet test`
- b. `uses:` serve solo per stampare stringhe
- ✓ c. `run:` esegue comandi di shell
- ✓ d. `uses:` richiama una action (es. `actions/checkout@v4`)
- e. sono degli alias, si possono usare in modo intercambiabile

Quali affermazioni su CLR e CTS sono corrette?

- a. CLR compila tutto sempre ahead-of-time e mai JIT
- b. CTS è un garbage collector
- ✓ c. CTS definisce regole/definizioni dei tipi che abilitano interoperabilità tra linguaggi .NET
- ✓ d. CLR è l'ambiente di esecuzione (runtime) per codice .NET

Perché esiste `IDisposable` in .NET?

- ✓ a. Serve per rilasciare deterministicamente risorse non gestite che sono incapsulate da oggetti .NET
- b. Ragioni storiche/di compatibilità: da C# 8 non serve più
- c. Per permettere agli sviluppatori di creare nuove classi

Quali affermazioni descrivono correttamente Git come DVCS (Distributed VCS)?

- a. Senza connessione Internet non si può fare `commit`
- b. Esiste un unico server centrale, GitHub, senza il quale il sistema non funziona
- ✓ c. Operazioni come `commit` e consultazione `log` sono tipicamente molto veloci perché avvengono in locale
- ✓ d. Ogni clone contiene l'intera cronologia del repository

Quali operazioni sono tipicamente possibili via reflection?

- a. Generare automaticamente unit-test
- ✓ b. Creare istanze con `Activator.CreateInstance(...)`
- ✓ c. Invocare metodi con `MethodInfo.Invoke(...)`
- ✓ d. Caricare assembly e scoprire tipi a runtime (plugin)
- e. Creare nuove istanze con `new`

In C#, i tipi "primitivi" (es. `int`) sono...

- a. indipendenti dal Common Type System
- ✓ b. alias di tipi del namespace `System` (es. `System.Int32`)
- c. tipi speciali che non hanno alcun corrispondente nel CLR
- d. sempre tipi riferimento

Quali problemi rientrano nel classico "it works on my machine"?

- ✓ a. Variabili d'ambiente/percorsi/credenziali non documentate
- ✓ b. Dipendenze implicite installate solo sulla macchina dello sviluppatore
- c. Il garbage collector non funziona in produzione
- d. Gli sviluppatori non hanno abbastanza spazio su disco
- e. Il continuo spostamento della data d'uscita di GTA 6

Quali affermazioni sono vere riguardo la *staging area (index)* in Git?

- ✓ a. Consente di costruire commit "pezzo per pezzo"
- ✓ b. Permette di selezionare quali modifiche includere nel prossimo commit
- c. È un backup automatico su GitHub
- d. Funziona solo in modalità online
- e. È obbligatoria solo nei repository molto grandi

Lavorando su un progetto .NET grande/complesso, perché è importante utilizzare un file di lock (`packages.lock.json`) insieme al file di progetto (`.csproj`)?

- a. Per non inserire troppe informazioni nel file `.csproj` che, in caso di molte dipendenze, diventerebbe troppo grande
- ✓ b. Il file di lock "congela" l'intero albero delle dipendenze, garantendo che la build sia riproducibile
- c. Facendo "lock" il codice generato è più efficiente
- ✓ d. Senza il lock file, una build potrebbe funzionare oggi ma "rompersi" domani
- ✓ e. Il file `.csproj` definisce solo le dipendenze dirette, ma non fissa le versioni esatte delle dipendenze transitive

Stai scrivendo un Dockerfile. Devi inserire i file del codice sorgente nell'immagine. Perché la documentazione e le best practice suggeriscono di usare `COPY . .` invece di `ADD . .` se non hai esigenze particolari?

- a. Perché `ADD` funziona solo in modalità rootless
- ✓ b. Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie
- c. Non c'è differenza, sono alias dello stesso comando
- d. Perché `COPY` crea un layer più piccolo rispetto ad `ADD`

Esegui `docker run -v ./data:/app/data my-image`. Quali affermazioni sono vere?

- [x] a. La cartella `./data` sull'host sovrascrive (oscura) il contenuto di `/app/data` nel container al momento dell'avvio
- [ ] b. Le cartelle `./data` e `/app/data` diventano read-only
- [ ] c. La cartella `/app/data` nel container sovrascrive (oscura) il contenuto di `./data` sull'host al momento dell'avvio

In un workflow GitHub Actions, hai tre job: `build`, `test` e `deploy`, definiti dalla configurazione seguente:

```
jobs:
  build:
    ...
  test:
    needs: build
    ...
  deploy:
    needs: build
    ...
```

Quali job verranno eseguiti e in che ordine?

- [ ] a. Sequenziale: `build`, poi `test`, poi `deploy`
- [ ] b. `build` parte per primo. Se finisce con successo, partirà `test`. Se finisce con successo, verrà eseguito `deploy`
- [x] c. `build` parte per primo. Se finisce con successo, `test` e `deploy` vanno in parallelo
- [x] d. Se `build` fallisce, `test` e `deploy` non verranno eseguiti
- [ ] e. `build`, `test` e `deploy` partono tutti contemporaneamente

In uno scenario open-source con fork, quali ruoli hanno tipicamente `origin` e `upstream`?

- [x] a. `upstream` è il repository originale
- [ ] b. `origin` è il repository originale
- [ ] c. `upstream` è il tuo fork
- [x] d. `origin` è il tuo fork

Che cos'è la **Reflection** in .NET e quali sono i suoi casi d'uso tipici?

- [x] a. Permette di caricare assembly di cui non si conosceva l'esistenza al momento della compilazione, abilitando architetture a *plugin*
- [ ] b. È una tecnica per ottimizzare le prestazioni: il codice che usa la Reflection è molto più veloce dell'accesso diretto ai metodi
- [x] c. È un meccanismo che permette a un programma di ispezionare i propri metadati (tipi, metodi, attributi) a runtime
- [x] d. Viene utilizzata dai framework di Test, come xUnit, per trovare ed eseguire i metodi marcati con attributi specifici

Sei sul branch `feature` e vuoi integrare le ultime modifiche fatte sul branch `main` (che è andato avanti nel frattempo). Decidi di usare `git rebase main`. Quali delle seguenti affermazioni descrivono correttamente cosa accade al tuo branch `feature`?

- [x] a. Git "stacca" i commit del tuo branch `feature` e li riapplica uno alla volta sopra gli ultimi commit di `main`
- [x] b. La storia risultante sarà lineare, come se avessi sviluppato la `feature` partendo dall'ultimo commit di `main`
- [ ] c. Questa operazione è sicura da eseguire anche se il branch `feature` è un branch pubblico condiviso su cui stanno lavorando altri colleghi
- [ ] d. Git crea un nuovo "Merge Commit" che unisce la storia di `main` e `feature`.

Hai avviato un container da un'immagine `ubuntu`. Mentre il container è in esecuzione, crei un file `/tmp/test.txt` al suo interno. Poi fermi il container con `docker stop`. Qual è lo stato del sistema?

- [ ] a. Il file `/tmp/test.txt` viene perso non appena il container si ferma
- [x] b. Se avvii un *nuovo* container dalla stessa immagine (`docker run ubuntu`), questo non conterrà il file `/tmp/test.txt`
- [ ] c. L'immagine originale `ubuntu` è stata modificata per includere il nuovo file.
- [x] d. Il file esiste ancora nel "container layer" scrivibile. Se riavvii lo stesso container (`docker start`), il file sarà lì.

Supponendo che `list` sia un'istanza di `List<int>`, come fa il compilatore C# a risolvere la chiamata `list.Where(x => x > 5)`, visto che `Where` non è un metodo definito nella classe `List<T>`?

- [ ] a. Cerca nelle sottoclassi di `List<T>` delle definizioni di metodi, chiamati `Where`, il cui primo parametro sia marcato con `this` e abbia un tipo compatibile con quello di `list`
- [ ] b. `Where` è un metodo virtuale di `System.Object` che viene ereditato da tutte le classi
- [ ] c. Il compilatore genera del codice che sfrutta la reflection, a runtime, per trovare metodi appropriati
- [x] d. Cerca nelle classi statiche le definizioni di metodi statici, chiamati `Where`, il cui primo parametro sia marcato con `this` e abbia un tipo compatibile con quello di `list`

Perché alcuni Dockerfile contengono più istruzioni `FROM`, per esempio:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
...
FROM mcr.microsoft.com/dotnet/runtime:9.0
COPY --from=build ...
```

Qual è lo scopo di questa tecnica?

a. Velocizzare il download dell'immagine permettendo il download parallelo dei layer

√ b. Ridurre la dimensione dell'immagine finale scartando gli strumenti di compilazione (SDK) e mantenendo solo il runtime e i binari compilati

c. Permettere al container di eseguire sia codice .NET 9 che codice legacy

d. Creare due immagini distinte con un solo comando `docker build`